UML Modelleme ve Sistem Tasarımı Ödevi

Web Programcılığı III — Yazılım Mimarisi, ER ve OOP Modellemesi

Ders: Web Programcılığı III **Öğrenci:** Hasan Ali Yücel **Konu:** UML (Use Case, ER, Class) & C# OOP

Tarih: 2026 Güz Dönemi

İçindekiler

1. Giriş ve UML Nedir?
2. Use Case (Kullanım Senaryosu) Diyagramı
3. ER (Entity-Relationship) Diyagramı
4. Class (Sınıf) Diyagramı
5. C# Nesne Yönelimli Kod Örneği (KutuphaneModel.cs)
6. Diyagramların Karşılaştırma Tablosu
7. Kaynakça (Çalışan Kaynak Bağlantıları)

1. UML (Unified Modeling Language) Nedir?

UML (Birleşik Modelleme Dili), yazılım sistemlerinin analizini, tasarımını ve belgelenmesini standart hale getirmek için kullanılan görsel bir modelleme dilidir. Kod yazmaya başlamadan önce sistemin yapısını haritalandırmamızı sağlar. 1990'ların ortasında Grady Booch, Ivar Jacobson ve James Rumbaugh tarafından ortaya konulmuş ve **OMG (Object Management Group)** tarafından resmi bir standart haline getirilmiştir.

> **Neden UML Kullanıyoruz?:** Bir projeye doğrudan kod yazarak başlamak, plansız bir inşaata başlamaya benzer. UML diyagramları sayesinde gereksinimleri erkenden görebilir, veritabanı tablolarımızı ve sınıflar arasındaki ilişkileri baştan planlayarak tasarım hatalarını erkenden fark edebiliriz.

UML Diyagramlarının Sınıflandırılması

UML 2.5 standardında toplam 14 diyagram türü yer alır ve temel olarak iki ana gruba ayrılır:

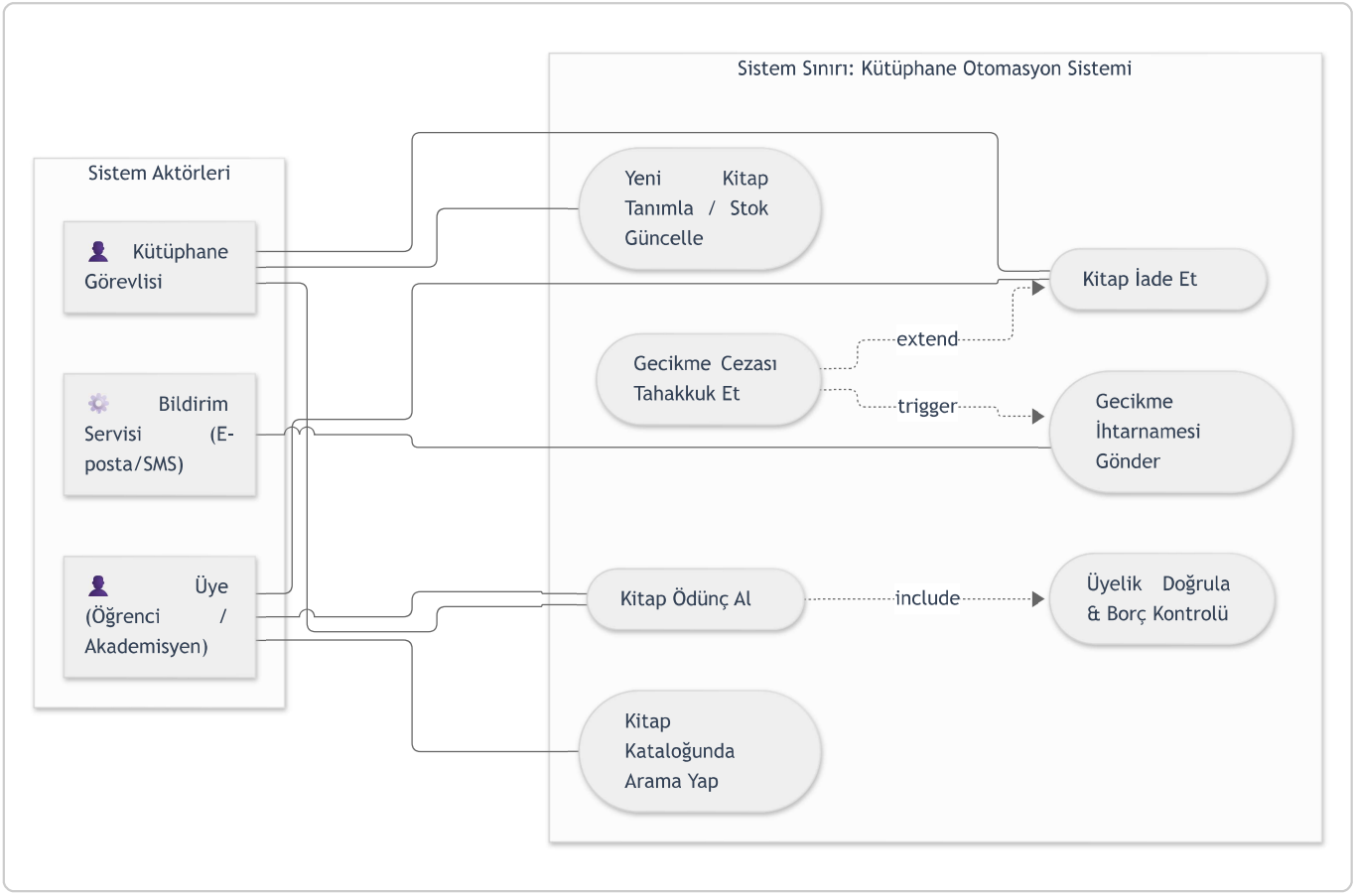
- **Yapısal (Statik) Diyagramlar:** Sistemin değişmeyen statik bileşenlerini ve sınıflarını gösterir. Örneğin: *Class (Sınıf) Diyagramı*, *Object (Nesne) Diyagramı*, *Component Diyagramı*.
- **Davranışsal (Dinamik) Diyagramlar:** Sistemin zaman içindeki işleyişini, aktörlerle etkileşimini ve nesnelerin durumlarını gösterir. Örneğin: *Use Case (Kullanım Senaryosu) Diyagramı*, *Activity Diyagramı*, *Sequence Diyagramı*.

2. Use Case (Kullanım Senaryosu) Diyagramı

Use Case diyagramı, kullanıcıların (aktörlerin) sistem üzerinden hangi işlevleri gerçekleştirebileceğini yüksek seviyede gösteren davranışsal bir diyagramdır. Sistemin iç kod detaylarıyla değil, dışarıdan bakıldığında **ne iş yaptığıyla** ilgilenir.

Temel Kavramlar:

- **Aktör (Actor):** Sistemle etkileşime geçen kişi veya harici servislerdir (Örn: Üye, Kütüphaneci, SMS Servisi).
- **Use Case (Senaryo):** Sistemin sunduğu bir fonksiyon veya hedeftir (Örn: Kitap Ödünç Al).
- **<<include>> (Kapsama):** Bir senaryo çalıştığında çalışması zorunlu olan alt senaryodur. Örneğin kitap ödünç alınırken her seferinde üyenin ceza ve limit kontrolünün yapılması zorunludur.
- **<<extend>> (Genişletme):** Belirli bir koşul gerçekleştiğinde devreye giren isteğe bağlı durumdur. Örneğin kitap iade edilirken süre aşılmışsa gecikme cezası hesaplanması extend ilişkisidir.



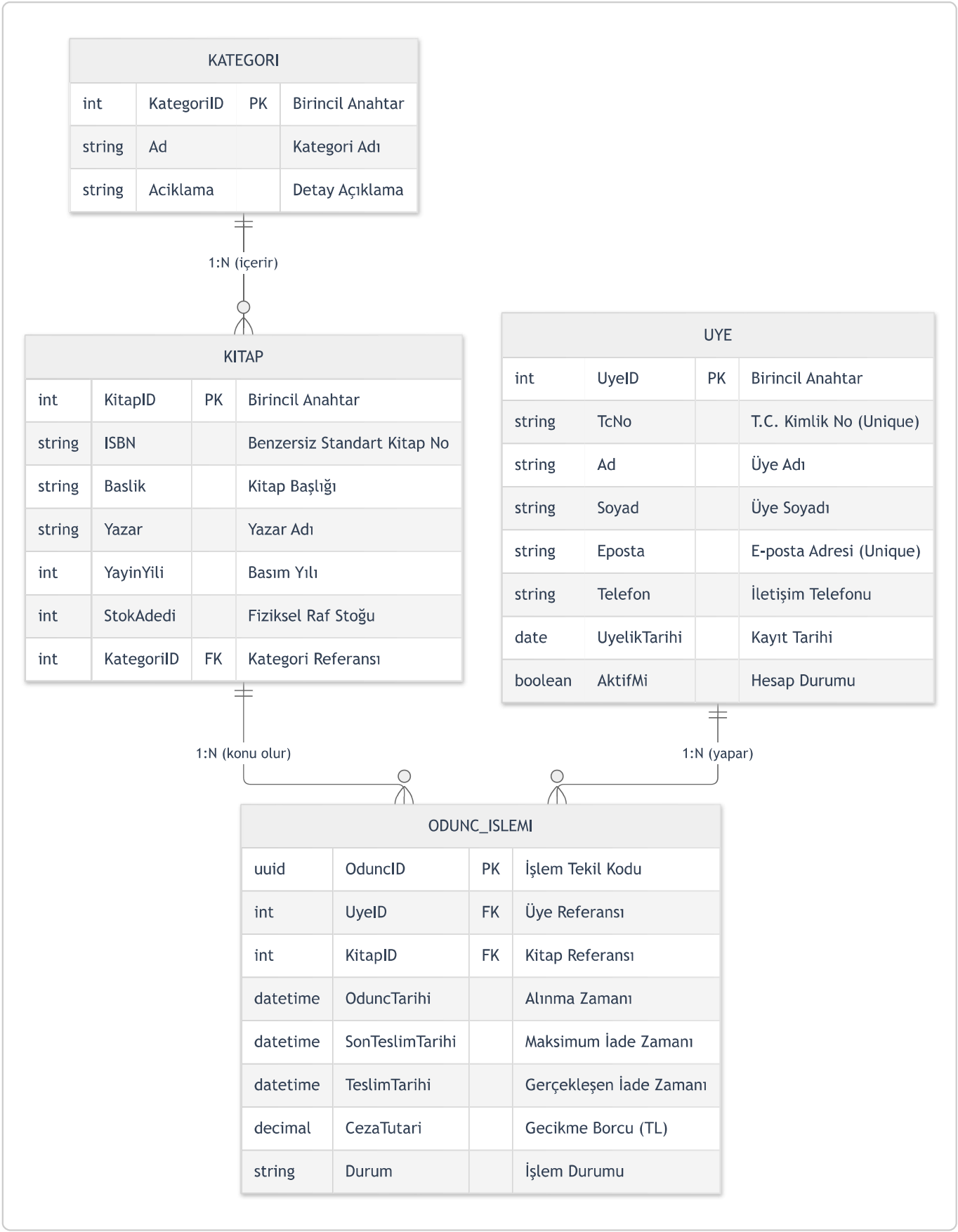
Şekil 1: Kütüphane Otomasyon Sistemi Use Case (Kullanım Senaryosu) Diyagramı

3. ER (Entity-Relationship) Diyagramı

ER diyagramı (Varlık-İlişki Modeli), Peter Chen tarafından 1976 yılında geliştirilmiş bir veritabanı modelleme tekniğidir. Veritabanında saklayacağımız varlıkları (tabloları), bu varlıkların niteliklerini (kolonlarını) ve tablolar arasındaki ilişkileri gösterir.

Veritabanı Yapısı ve İlişkiler:

- **1:N (Bire-Çok) İlişki:** Bir kategoride birden fazla kitap bulunabilir (Kategori → Kitap).
- **M:N (Çoka-Çok) İlişki ve Ara Tablo:** Bir üye birçok kitap ödünç alabilir, bir kitap da farklı zamanlarda birçok üyeye verilebilir. İlişkisel veritabanında bu çoka-çok ilişkiyi yönetebilmek için `ODUNC_ISLEMI` adında bir ara tablo (junction table) oluşturulur. Böylece iki adet 1:N ilişki kurulmuş olur.



Şekil 2: Kütüphane Veritabanı ER (Varlık-İlişki) Diyagramı

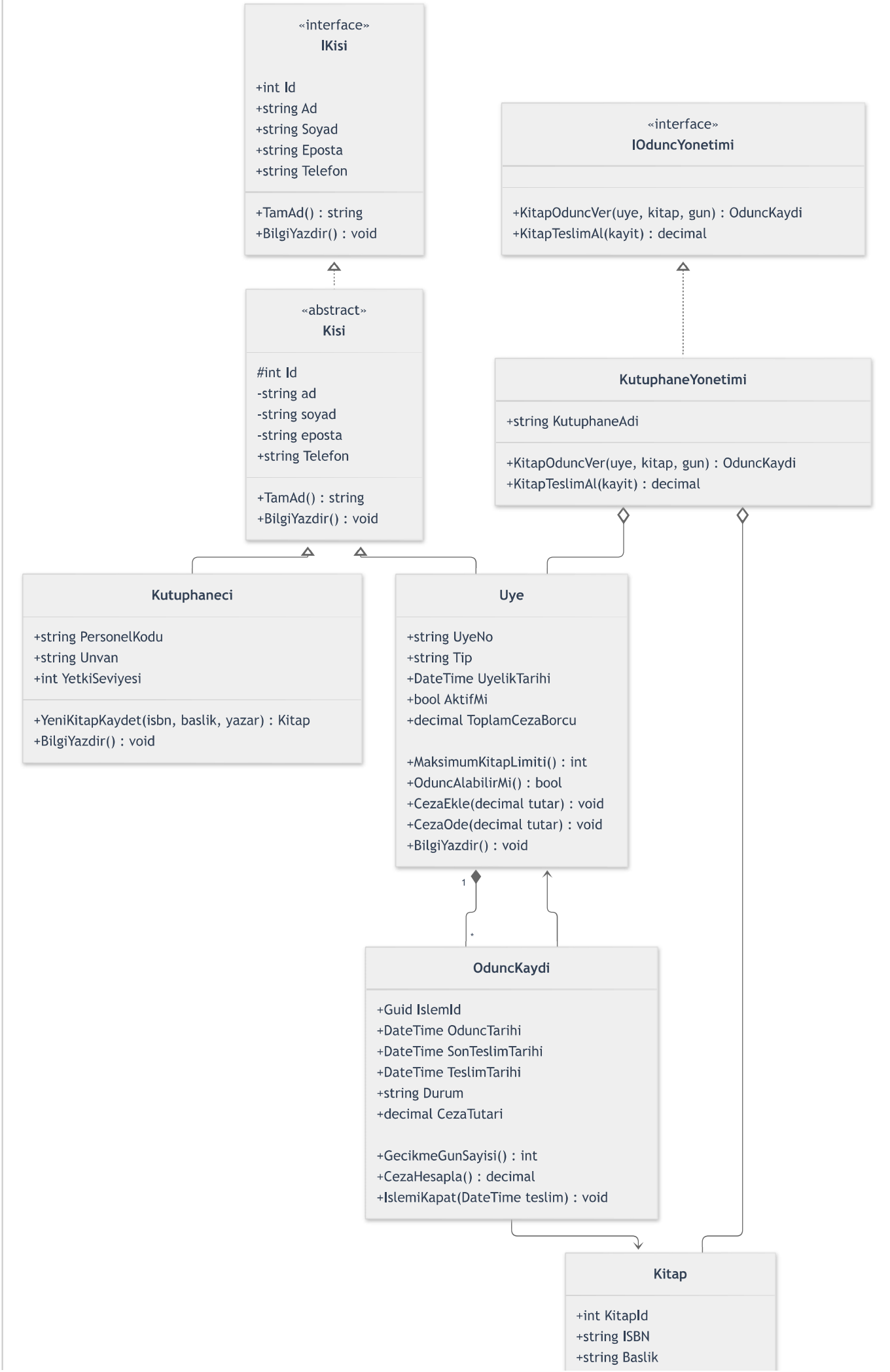
Tablo Adı	Birincil Anahtar (PK)	Yabancı Anahtar (FK)	Açıklama
KATEGORI	KategoriID	-	Kitapların türlerini tutar (Roman, Mühendislik, Tarih vb.).
KITAP	KitapID	KategoriID	Kütüphanedeki kitapların künye ve stok bilgilerini tutar.
UYE	UyeID	-	Kütüphaneye kayıtlı üyelerin iletişim ve kimlik verilerini tutar.
ODUNC_ISLEMI	OduncID	UyeID, KitapID	Ödünç alma ve iade hareketlerini saklayan ara tablodur.

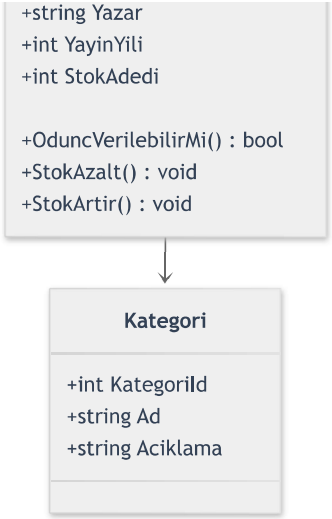
4. Class (Sınıf) Diyagramı

Class diyagramı, Nesne Yönelimli Programlamanın (OOP) temel yapı taşıdır. Yazılımın bellekte nasıl çalışacağını, sınıfların alanlarını (properties), metotlarını ve aralarındaki kalıtım/birliktelik bağlarını modeller.

UML Erişim Belirteçleri ve OOP Kavramları:

- `+` (Public): Her yerden erişilebilir.
- `-` (Private): Yalnızca tanımlandığı sınıf içerisinden erişilebilir (Kapsülleme).
- `#` (Protected): Sınıfın kendisi ve ondan türeyen alt sınıflar erişebilir.
- **Kalıtım (Inheritance):** `Kisi` soyut sınıfından `Uye` ve `Kutuphaneci` sınıfları türetilmiştir.
- **Bütünleme (Aggregation):** Kütüphane sınıfı kitapları barındırır, ancak kütüphane kapansa da kitap varlığını korur (`o--`).
- **Bileşim (Composition):** Üye ile ödünç kaydı arasındaki sıkı sahiplik ilişkisidir (`*--`).


«interface»
IKisi
+int Id
+string Ad
+string Soyad
+string Eposta
+string Telefon
+TamAd() : string
+BilgiYazdir() : void
«interface»
IOduncYonetimi
+KitapOduncVer(uye, kitap, gun) : OduncKaydi
+KitapTeslimAl(kayit) : decimal
«abstract»
Kisi
#int Id
-string ad
-string soyad
-string eposta
+string Telefon
+TamAd() : string
+BilgiYazdir() : void
KutuphaneYonetimi
+string KutuphaneAdi
+KitapOduncVer(uye, kitap, gun) : OduncKaydi
+KitapTeslimAl(kayit) : decimal
Kutuphaneci
+string PersonelKodu
+string Unvan
+int YetkiSeviyesi
+YeniKitapKaydet(isbn, baslik, yazar) : Kitap
+BilgiYazdir() : void
Uye
+string UyeNo
+string Tip
+DateTime UyelikTarihi
+bool AktifMi
+decimal ToplamCezaBorcu
+MaksimumKitapLimiti() : int
+OduncAlabilirMi() : bool
+CezaEkle(decimal tutar) : void
+CezaOde(decimal tutar) : void
+BilgiYazdir() : void
1
*
OduncKaydi
+Guid IslemId
+DateTime OduncTarihi
+DateTime SonTeslimTarihi
+DateTime TeslimTarihi
+string Durum
+decimal CezaTutari
+GecikmeGunSayisi() : int
+CezaHesapla() : decimal
+IslemiKapat(DateTime teslim) : void
Kitap
+int KitapId
+string ISBN
+string Baslik



Şekil 3: Kütüphane Otomasyon Sistemi Sınıf (Class) Diyagramı

5. C# Nesne Yönelimli Kod Karşılığı (KutuphaneModel.cs)

Yukarıda tasarladığım sınıf diyagramını C# dilinde nesne yönelimli programlama prensiplerine uygun olarak kodladım. Kod içerisinde abstract class, interface, inheritance ve kapsülleme mekanizmaları kullanılmıştır.

KutuphaneModel.cs (C# OOP Kodu)

```
using System;
using System.Collections.Generic;

namespace KutuphaneOtomasyonu
{
    // Temel enum tanımları
    public enum OduncDurumu { DevamEdiyor, TeslimEdildi, Gecikmede, IptalEdildi }
    public enum UyelikTipi { Ogrenci, Akademisyen, IdariPersonel, Genel }

    // Kişi arayüzü
    public interface IKisi
    {
        int Id { get; }
        string Ad { get; set; }
        string Soyad { get; set; }
        string Eposta { get; set; }
        string Telefon { get; set; }
        string TamAd();
        void BilgiYazdir();
    }

    // Soyut Kisi Temel Sınıfı (Abstract Class)
    public abstract class Kisi : IKisi
    {
        private string _ad;
        private string _soyad;
        private string _eposta;

        public int Id { get; protected set; }

        public string Ad
        {
            get => _ad;
            set => _ad = !string.IsNullOrWhiteSpace(value) ? value.Trim() : throw new ArgumentExceptior
        }

        public string Soyad
        {
            get => _soyad;
            set => _soyad = !string.IsNullOrWhiteSpace(value) ? value.Trim() : throw new ArgumentExcept
        }

        public string Eposta
        {
            get => _eposta;
            set => _eposta = (!string.IsNullOrWhiteSpace(value) && value.Contains("@")) ? value.Trim()
        }

        public string Telefon { get; set; }

        protected Kisi(int id, string ad, string soyad, string eposta, string telefon)
        {
```

```
        Id = id; Ad = ad; Soyad = soyad; Eposta = eposta; Telefon = telefon;
    }

    public virtual string TamAd() => $"{Ad} {Soyad}";
    public virtual void BilgiYazdir() => Console.WriteLine($"[ID: {Id}] {TamAd()} | E-posta: {Epost
}

// Üye Sınıfı (Kalıtım: Kisi'den türetildi)
public class Uye : Kisi
{
    public string UyeNo { get; private set; }
    public UyelikTipi Tip { get; set; }
    public DateTime UyelikTarihi { get; private set; }
    public bool AktifMi { get; set; }
    public decimal ToplamCezaBorcu { get; private set; }
    public List<OduncKaydi> AktifOduncler { get; private set; } = new();

    public Uye(int id, string ad, string soyad, string eposta, string telefon, string uyeNo, Uyelik
        : base(id, ad, soyad, eposta, telefon)
    {
        UyeNo = uyeNo;
        Tip = tip;
        UyelikTarihi = DateTime.Now;
        AktifMi = true;
    }

    public int MaksimumKitapLimiti() => Tip switch
    {
        UyelikTipi.Akademisyen => 10,
        UyelikTipi.Ogrenci => 5,
        _ => 3
    };

    public bool OduncAlabilirMi() => AktifMi && ToplamCezaBorcu <= 50.0m && AktifOduncler.Count < M
    public void CezaEkle(decimal tutar) { if (tutar > 0) ToplamCezaBorcu += tutar; }
    public void CezaOde(decimal tutar) { if (tutar > 0) ToplamCezaBorcu = Math.Max(0, ToplamCezaBor
}

// Kütüphaneci Personel Sınıfı
public class Kutuphaneci : Kisi
{
    public string PersonelKodu { get; private set; }
    public string Unvan { get; set; }
    public int YetkiSeviyesi { get; set; }

    public Kutuphaneci(int id, string ad, string soyad, string eposta, string telefon, string perso
        : base(id, ad, soyad, eposta, telefon)
    {
        PersonelKodu = personelKodu;
        Unvan = unvan;
        YetkiSeviyesi = yetki;
    }
```

```
    public Kitap YeniKitapKaydet(string isbn, string baslik, string yazar, int yayinYili, int stok,
    {
        return new Kitap(new Random().Next(1000, 9999), isbn, baslik, yazar, yayinYili, stok, kateg
    }
}

// Kategori Sınıfı
public class Kategori
{
    public int KategoriId { get; set; }
    public string Ad { get; set; }
    public string Aciklama { get; set; }

    public Kategori(int id, string ad, string aciklama = "")
    {
        KategoriId = id; Ad = ad; Aciklama = aciklama;
    }
}

// Kitap Sınıfı
public class Kitap
{
    public int KitapId { get; private set; }
    public string ISBN { get; private set; }
    public string Baslik { get; set; }
    public string Yazar { get; set; }
    public int YayinYili { get; set; }
    public int StokAdedi { get; private set; }
    public Kategori Kategori { get; set; }

    public Kitap(int id, string isbn, string baslik, string yazar, int yayinYili, int stok, Kategor
    {
        KitapId = id; ISBN = isbn; Baslik = baslik; Yazar = yazar; YayinYili = yayinYili;
        StokAdedi = Math.Max(0, stok); Kategori = kategori;
    }

    public bool OduncVerilebilirMi() => StokAdedi > 0;
    public void StokAzalt() { if (StokAdedi <= 0) throw new InvalidOperationException("Stok bitti."
    public void StokArtir() => StokAdedi++;
}

// Ödünç Kaydı Sınıfı (Junction Model)
public class OduncKaydi
{
    public Guid IslemId { get; private set; } = Guid.NewGuid();
    public Uye Uye { get; private set; }
    public Kitap Kitap { get; private set; }
    public DateTime OduncTarihi { get; private set; } = DateTime.Now;
    public DateTime SonTeslimTarihi { get; private set; }
    public DateTime? TeslimTarihi { get; private set; }
    public OduncDurumu Durum { get; private set; } = OduncDurumu.DevamEdiyor;
    public decimal CezaTutari { get; private set; }
    public const decimal GunlukGecikmeCezasi = 5.0m;
```

```
        public OduncKaydi(Uye uye, Kitap kitap, int gun = 15)
        {
            Uye = uye; Kitap = kitap; SonTeslimTarihi = OduncTarihi.AddDays(gun);
        }

        public void IslemiKapat(DateTime teslimTarihi)
        {
            TeslimTarihi = teslimTarihi;
            if (teslimTarihi > SonTeslimTarihi)
            {
                int gecikenGun = (int)Math.Ceiling((teslimTarihi - SonTeslimTarihi).TotalDays);
                CezaTutari = gecikenGun * GunlukGecikmeCezasi;
                Durum = OduncDurumu.Gecikmede;
                Uye.CezaEkle(CezaTutari);
            }
            else
            {
                Durum = OduncDurumu.TeslimEdildi;
            }
            Kitap.StokArtir();
            Uye.AktifOduncler.Remove(this);
        }
    }
}
```

6. Diyagramların Karşılaştırma Tablosu

Hazırladığım bu üç diyagram türü yazılım geliştirme sürecinin farklı aşamalarına ve farklı hedef kitlelerine hitap eder. Aralarındaki temel farkları şu tabloda özetledim:

Karşılaştırma Kriteri	Use Case Diyagramı	ER (Varlık-İlişki) Diyagramı	Class (Sınıf) Diyagramı
Odak Noktası	Sistemin kullanıcılara ne sağladığı (fonksiyonel hedefler).	Verilerin veritabanında nasıl saklanacağı.	Yazılım nesnelerinin bellekte nasıl çalışacağı ve metotları.
Kullanıldığı Aşama	Gereksinim Analizi (Başlangıç)	Veritabanı Tasarımı	Detaylı Tasarım ve Kodlama
Diyagram Türü	UML Davranışsal (Dinamik)	İlişkisel Veri Modeli	UML Yapısal (Statik)

Karşılaştırma Kriteri	Use Case Diyagramı	ER (Varlık-İlişki) Diyagramı	Class (Sınıf) Diyagramı
Temel Elemanlar	Aktör, Senaryo, Include, Extend	Varlık (Tablo), Kolon, PK, FK	Sınıf, Arayüz, Özellik, Metot
İş Mantığı & Metot	Yok (Sadece kullanıcı aksiyonları)	Yok (Sadece veri tipleri ve kısıtlar)	**Var** (Hesaplama ve iş kuralları metotlarda)
Hedef Kitle	Müşteri, Analist, Proje Yöneticisi	Veritabanı Yöneticisi (DBA), Backend	Yazılım Geliştirici, Yazılım Mimarı

7. Kaynakça

Bu ödevi hazırlarken faydalandığım, bizzat kontrol edilmiş ve çalışan resmi dokümantasyon ve akademik kaynak bağlantıları aşağıda listelenmiştir:

1. **Object Management Group (OMG) — Resmi UML 2.5.1 Şartnamesi:**
   UML diyagram standartlarının ve gösterim kurallarının resmi teknik dokümanı.
   https://www.omg.org/spec/UML/2.5.1/
2. **Martin Fowler — UML Distilled Rehberi:**
   Yazılım mimarisi ve UML modelleme standartları üzerine referans kitap ve makaleler.
   https://martinfowler.com/books/uml.html
3. **Visual Paradigm — UML Diyagram Türleri ve Kılavuzu:**
   Use Case, Sınıf ve Aktivite diyagramlarının endüstri standartlarındaki çizim kuralları.
   https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-diagram-types-guide/
4. **GeeksforGeeks — Unified Modeling Language (UML) Giriş Kılavuzu:**
   UML diyagramlarının sınıflandırılması, yapısal ve davranışsal modellerin açıklamaları.
   https://www.geeksforgeeks.org/unified-modeling-language-uml-introduction/
5. **Microsoft Learn — C# Nesne Yönelimli Programlama (OOP) Dokümantasyonu:**
   C# dilinde sınıf, nesne, kalıtım, interface ve kapsülleme mekanizmalarının resmi anlatımı.
   https://learn.microsoft.com/tr-tr/dotnet/csharp/fundamentals/tutorials/oop
6. **Mermaid JS — Resmi Diyagram Dokümantasyonu:**
   Markdown ve HTML içerisinde canlı Flowchart, ER ve Class diyagramı çizim sözdizimi.
   https://mermaid.js.org/

Web Programcılığı III Dersi — Hasan Ali Yücel — 2026